2520090063
R.Radhakrishna

Practical Tasks

Task-3

Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution

```
Radhakrishna@04:~#nano task3.c
Radhakrishna@04:~#
```

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Before fork():\n");
    printf("Process PID: %d\n", getpid());
    printf("Parent PID: %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }
```

```
if (pid == 0) {
    printf("CHILD PROCESS\n");
    printf("PID  : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("State: Running\n");

    sleep(5);

    printf("\nChild process after sleep\n");
    printf("PID  : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("State: Running\n");

    printf("Child process terminating...\n");
    exit(0);
}
else {
    printf("PARENT PROCESS\n");
    printf("PID  : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("Child PID: %d\n", pid);
    printf("State: Running\n");

    printf("\nParent waiting for child...\n");
    wait(NULL);

    printf("Child process completed.\n");
    printf("Parent process terminating...\n");
}
```

```
    return 0;
}
```

```
Radhakrishna@04:~#nano task3.c
Radhakrishna@04:~#.task3
```

Output

```
Before fork():
Process PID: 575
Parent PID: 316

PARENT PROCESS
PID  : 575
PPID : 316
Child PID: 576
State: Running

Parent waiting for child...
CHILD PROCESS
PID  : 576
PPID : 575
State: Running

Child process after sleep
PID  : 576
PPID : 575
State: Running
```

```
Child process terminating...
Child process completed.
Parent process terminating...
```

Task-4

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

```
Radhakrishna@04:~#nano task4.c
Radhakrishna@04:~#
```

Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <time.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process Started\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());
```

```
    printf("\nState 1: WAITING/SLEEPING\n");
    sleep(10);

    printf("\nState 2: RUNNING\n");

    time_t start = time(NULL);

    while (time(NULL) - start < 15) {
        volatile unsigned long long x = 0;
        for (unsigned long long i = 0; i < 10000000; i++) {
            x += i;
        }
    }

    printf("\nState 3: TERMINATING\n");
    exit(0);
}
else {
    printf("Parent Process\n");
    printf("PID       : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    printf("\nParent is waiting for 35 seconds.\n");
    sleep(35);

    printf("\nParent calling wait().\n");
    wait(NULL);

    printf("Child process collected.\n");
```

```
        printf("Parent terminating.\n");
    }

    return 0;
}
```

```
Radhakrishna@04:~#nano task4.c
Radhakrishna@04:~# gcc task4.c -o task4
Radhakrishna@04:~#./task4
```

Output

```
Parent Process
PID       : 677
Child PID : 678
Parent is waiting for 35 seconds.
Child Process Started
PID  : 678
PPID : 677
State 1: WAITING/SLEEPING
State 2: RUNNING
State 3: TERMINATING
Parent calling wait().
Child process collected.
Parent terminating.
```